

Questão 1 (Complexidades, 2.5pt)

Para os algoritmos a seguir, liste suas complexidades e forneça uma breve explicação de como eles são obtidos (i.e. o que explica os diferentes termos da complexidade).

Dijkstra, Edmonds-Karp, Caminho mais gordo, Hopcroft-Karp, Algoritmo Húngaro.

Dijkstra $O((n+m) \log n)$ usando um heap binário: temos n operações insert e deletemin, e m operações update, cada uma de custo $O(\log n)$.

Edmonds-Karp $O(nm^2)$: temos no máximo nm fases, cada fase uma busca em largura de custo $O(n + m)$.

Caminho mais gordo : $O(Dm \log C)$: temos no máximo $m \log C$ iterações, C sendo um limitante superior ao fluxo máximo, e cada iteração aplica o algoritmo de Dijkstra em tempo D ($D = O(n \log n + m)$).

Hopcroft-Karp $O(n^{5/2})$: temos no máximo $\sqrt{n}$ fases, cada uma uma busca em largura de tempo $O(n + m)$, logo $O((n + m)\sqrt{n}) = O(m\sqrt{n}) = O(n^{5/2})$.

Algoritmo Húngaro $O(nD)$, temos no máximo $O(n)$ fases, cada uma uma execução do algoritmo de Dijkstra em tempo D .

Questão 2 (Caminhos mais curtos com distâncias negativas, 2.5pt)

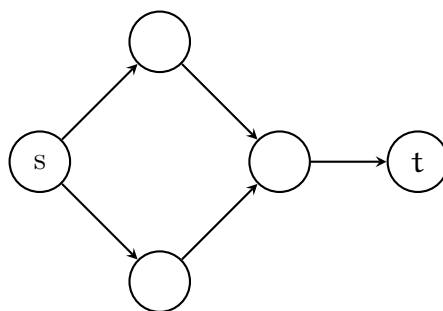
- Sob quais condições os algoritmos de Bellman-Ford e Johnson produzem resultados corretos?
- No contexto do algoritmo de Johnson, explique a ideia de potenciais, como eles podem ser obtidos e o que eles alcançam.
- Caso não existem ciclos de custo negativo.
- Um potencial é uma função $p : V \rightarrow \mathbb{R}$, tal que $d_{uv} \geq p_v - p_u$. Logo $d'_{uv} = d_{uv} - (p_v - p_u) \geq 0$ são distâncias não-negativas. A

importância é que um caminho sv mais curto em d também é mais curto em d' , logo na posse de um potencial podemos aplicar um algoritmo de caminhos mais curto com distâncias não-negativas, que é mais eficiente.

Questão 3 (Fluxo máximo, 2.5pt)

Considere o problema do fluxo máximo num grafo direcionado capacitado entre dois vértices s e t .

- a) Descreva brevemente, mas de forma precisa, o conteúdo do teorema do corte mínimo do fluxo máximo.
 - b) Dê um exemplo em que temos vários fluxos máximos diferentes (obviamente de mesmo valor), mas um único corte mínimo, ou mostra que isso é impossível.
 - c) Dê um exemplo em que temos um único fluxo máximo, mas vários cortes mínimos, ou mostra que isso é impossível.
 - d) Pergunta bônus: se um grafo tiver vários cortes mínimos, qual corte mínimo devolvem os algoritmos do tipo Ford-Fulkerson? Caracterize os cortes devolvidos.
- a) Seja X qualquer corte e f qualquer fluxo, temos $f(s) \leq c(X)$. Caso X^* é um corte mínimo e f^* um fluxo máximo, ainda $f(s) = c(X)$.
 - b) É simples de ver que o grafo



com todos arcos de capacidade unitária, possui 2 fluxos máximos (inteiros, ou um número infinito de fluxos máximos reais), ambos de valor 1, mas somente um corte mínimo.

c) O grafo



(novamente todos arcos de capacidade unitária) possui um fluxo máximo, de valor 1, mas dois cortes mínimos.

d) Assume primeiramente que existem somente dois cortes st mínimos C_1, C_2 (onde $s \in C_1, s \in C_2$). Caso $C_1 \subseteq C_2$ então algoritmos do tipo FF retornam C_1 : isso porque o corte retornado é por busca em largura a partir de s no grafo residual e não tem arcos que saem de C_1 , logo um vértice em $C_2 \setminus C_1$ não é alcançável. Agora caso $C_1 \oplus C_2 \neq \emptyset$ (ou seja nem $C_1 \subseteq C_2$ nem $C_2 \subseteq C_1$) o conjunto $C = C_1 \cap C_2$ é outro corte: $s \in C$, porque $s \in C_1$ e $s \in C_2$, e não tem arco sainte de C , porque tal arco tem que sair de C_1 ou C_2 .

Isso implica que no caso geral $\mathcal{C} = \{C \mid C \text{ é corte } st \text{ mínimo}\}$ tem um elemento mínimo $C_{\min}$ bem definido, a saber $C_{\min} = \bigcap_{C \in \mathcal{C}} C$ tal que $C_{\min} \subseteq C$, para todo $C \in \mathcal{C}$. Algoritmos do tipo FF retornam $C_{\min}$.

Questão 4 (Algoritmos randomizados, 2.5pt)

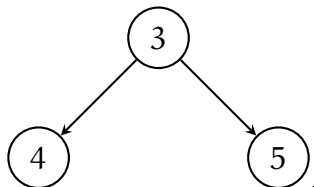
Você tem um algoritmo de decisão do tipo Las Vegas que executa em tempo polinomial, mas por um problema na sua máquina de execução, cada resposta é invertida com probabilidade $p = 0.1$. Forneça uma estratégia para reduzir a probabilidade de erro para menos de 0.03. Demonstra o resultado.

Isso é uma aplicação simples da amplificação de probabilidades: roda o algoritmo três vezes, e retorna a resposta majoritária. A probabilidade de falhar (nos dois lados) é $3 \cdot 0.1^2 0.9 + 0.1^3 = 0.028 \leq 0.03$.

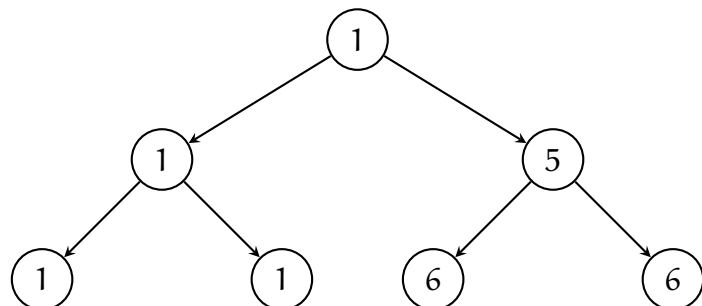
Questão 5 (Heaps, 2.5pt)

Uma colega tem três ideias para fazer com que os heaps funcionem melhor. Nesta pergunta, assumiremos min heaps. Elas funcionam? (Prova ou contra-exemplo.)

- a) Suponha que implementemos o deletemin com um aumento para ∞ seguido de heapify-down – que torna a chave uma folha – e, em seguida, removendo a chave. Isso funciona?
 - b) Ela também observa que sempre há a possibilidade de permutar os irmãos (juntamente com suas subárvores), já que, pela propriedade de heap, as chaves dos pais são sempre menores. Isso pode ser usado, por exemplo, para garantir que a chave do filho da esquerda seja sempre menor do que a do filho da direita.
 - c) Finalmente ela observa que sempre há a possibilidade de permutar todas as folhas, por um argumento similar ao item b).
- a) Não funciona: ∞ vira folha, mas não a última folha. Não podemos simplesmente remover sem desbalancear o heap (e por consequência perder as garantias de complexidade). Um contra-exemplo simples é



- b) Sim é possível. Considera um nó p com descendentes $l = l(p)$ e $r = r(p)$. Sobre uma troca $l(p) := r$ e $r(p) := l$ a propriedade do heap está mantida.
- c) Não funciona. Um contra-exemplo é



onde a permutação de folhas (6 6 1 1) não satisfaz a propriedade do heap.

Questão 6 (Caminhos mais curtos, 2.5pt)

Um colega afirma que o algoritmo de Dijkstra não apenas calcula as distâncias mais curtas para todos os vértices a partir de um vértice de origem s , mas também pode ordenar os vértices em ordem crescente de distâncias (por exemplo, por guardar em um vetor em ordem de processamento). Portanto, ele conclui que um tal algoritmo tem complexidade de pelo menos $\Omega(n \log n)$. Ele está certo? Discuta.

Está. O algoritmo proposto funciona, e pode ser implementado em custo $O(1)$ por elemento adicionado ao vetor, logo sem custo efetivo adicional no algoritmo de Dijkstra. Mas qualquer algoritmo que usa somente comparações precisa tempo $\Omega(n \log n)$ para ordenar. Então não pode existir um algoritmo mais rápido que além das distâncias também fornece os vértices em ordem das distâncias. Senão podemos ordenar $a_1, a_2, \dots, a_n$ em tempo menos que $O(n \log n)$, por criar um grafo $G = ([0, n], \{0i \mid i \in [n]\})$ com $d_{0i} = a_i$ e rodar tal algoritmo com vértice origem 0.

(É possível construir algoritmos de caminhos mais curtos mais rápidos que somente produzem as distâncias, sem ordenar, e.g. Duan et al. (2025).)

Referências

- [1] Ran Duan et al. “Breaking the Sorting Barrier for Directed Single-Source Shortest Paths”. Em: (23 de abr. de 2025). DOI: [10.48550/ARXIV.2504.17033](https://doi.org/10.48550/ARXIV.2504.17033). arXiv: [2504.17033](https://arxiv.org/abs/2504.17033) [cs.DS].